PROJECT-BASED LEARNING

PHASE-I EVALUATION

Maze Lab: Interactive Maze Generation and Comparative Pathfinding

Team ID: T272 | Team Blockbuster

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: T272 (Team Blockbuster)
Semester: 3rd Semester
Course: DSCPP-III-2026-T272
Domain: Graph Algorithms & OOP

Team Members

1. Tanmay Kumar (Lead) – 2510370439
2. Sparsh Saxena – 2510370313
3. Hemang Sethi – 2510370341

Mentor

Mr. Sachin Bhatt

Interactions completed: 3 Sessions

Problem Statement & Motivation

Problem Statement

Data structures and algorithms are typically taught as standalone academic topics. Students struggle to visualize how graph traversals, priority queues, and union-find structures operate dynamically within a single, unified software system.

Motivation

- Abstract graph concepts are hard to visualize.
- Static textbook diagrams lack runtime traces.
- Current solutions rarely combine player gameplay with pathfinding benchmarking.

Target Users / Stakeholders

- Computer Science students studying DSA and OOP.
- Instructors demonstrating algorithm efficiency.
- Software developers analyzing search heuristics.

Objectives

Project Objectives

1. Build a modular C++ system for generating perfect mazes dynamically.
2. Implement interactive player navigation with boundary collision logic.
3. Construct automated solvers using BFS and A* heuristic pathfinding.
4. Record evaluation metrics including path distance, evaluated nodes, and time.
5. Demonstrate object-oriented strategy patterns for plug-and-play algorithm design.

Key Objective: Compare human navigation efficiency directly against BFS and A* graph algorithms on identical grid layouts.

Proposed Solution

Proposed Approach

1. **Grid Setup:** 2D grid of `Cell` structures.
2. **Maze Generation:** Iterative DFS & Kruskal's algorithm.
3. **Player Mechanics:** Terminal character controls.
4. **Graph Solving:** BFS and A* with Manhattan distance.
5. **Metrics Engine:** Collect runtime and traversal stats.

Key Features

- Modular strategy interfaces for generators/solvers.
- Stack-based DFS to prevent call-stack overflow.
- Disjoint Set Union (DSU) cycle prevention.
- Real-time terminal rendering of player & path.
- Side-by-side performance benchmarking.

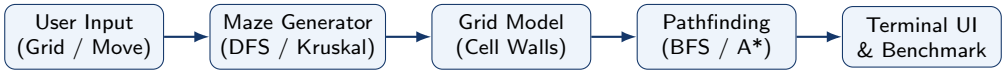
Provides a practical platform for exploring theoretical graph structures through visual interaction.

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C / C++	Explicit Stacks, Queues, Priority Queues, Disjoint Set Union (DSU), 2D Arrays	Maze Generators (DFS, Kruskal) & Solvers (BFS, A*)
OOPs with C++	Classes, Abstract Interfaces, Inheritance, Encapsulation, Strategy Pattern	`Maze`, `Cell`, `MazeGenerator`, `MazeSolver`, `Player`

Requirement: Demonstrates real-time integration of C++ object-oriented design and fundamental graph algorithms in a single system.

System Architecture / Workflow



Major Components

- `Maze` & `Cell`: Grid ownership and wall state.
- `MazeGenerator`: Abstract generation strategy.
- `MazeSolver`: Abstract pathfinding strategy.
- `Player`: Movement & collision detection.

Data / Control Flow

- User defines dimensions $\rightarrow$ Maze generated.
- Player traverses grid via key commands.
- Solvers evaluate grid as undirected graph.
- Execution metrics output to terminal.

Technology Stack & Methodology

Technology Stack

- **Language:** C++ (ISO C++17 Standard)
- **Data Structures:** `std::vector`, `std::priority_queue`, `std::stack`, Custom DSU
- **Build Tool:** g++ / CMake
- **Version Control:** Git & GitHub (`https://github.com/Tanmay-git65/III-SEM-PBL`)
- **Optional Graphics:** SDL2 (Stretch Goal)

Development Methodology

1. Architecture & Strategy Interface Design
2. Core Grid & DFS Generation
3. Terminal Display & Collision Engine
4. BFS & A* Solver Implementation
5. Benchmarking & Evaluation

Selected C++17 for zero-overhead abstractions and direct control over memory structures.

Initial Progress & Team Contribution

Progress Achieved (Phase-I)

- System design & UML architecture complete.
- `Cell` and `Maze` models implemented.
- Iterative DFS generator working.
- Player movement and collision active.
- BFS pathfinder with parent maps operational.

Individual Contribution

Member	Role	Contribution
Tanmay Kumar	Lead / Arch	Grid Architecture, Interfaces, Execution Loop
Sparsh Saxena	Core Dev	Iterative DFS Generator, BFS Solver, A* Pathfinder
Hemang Sethi	Dev / UI	Player Movement, Collision Checks, Terminal UI, DSU

Roadmap, Expected Outcomes & References

Roadmap

1. **Phase-I:** System Design, DFS, BFS (Done)
2. **Phase-II:** A*, Kruskal's, Metrics (In Progress)
3. **Phase-III:** Comparative Analysis & GUI Demo

Expected Outcomes

- Fully functional C++ terminal application.
- Measurable timing & node traversal comparison.
- Modular, extensible algorithm framework.

References

1. Cormen, T. H., et al. *Introduction to Algorithms*. MIT Press.
2. Gamma, E., et al. *Design Patterns*. Addison-Wesley.
3. ISO/IEC C++17 Standard Specifications.

Thank You
Questions & Suggestions